A* by Smashmouth

Miles Mena, Nick McConnell, Spencer Mullinix, Jonathan Wu

***Abstract*— While the field of robotics is beginning to incorporate foundation models and other generative techniques, there still exists a vital need for well-tested, robust robotic methods. The AWS DeepRacer platform offers reliable hardware with enough computation to tackle a variety of challenges. In this report, we detail method's and approaches implemented to solve the chosen challenges in completion of CSCI 4302-5302-001 at the University of Colorado - Boulder. The code for the ROS packages can be found at: https://github.com/mullisd1/A-by-Smashmouth/tree/dev/intimidation_mode and the code for the depth estimation server can be found at: https://github.com/jonathancsci/depth-estimation-server.**

I. INTRODUCTION

The AWS DeepRacer is a 1/8th scale race car intended for developers to apply machine learning techniques as they apply to robotics. AWS has hosted international competitions, where teams can deploy reinforcement models from a disk-on-key to the DeepRacer platform. While the main thrust of the AWS DeepRacer challenge is simulation, the platform provides the oppurtunity to mix essential robotic methods with state of the art machine learning techniques.

II. CHALLENGES

A. Intimidation Mode

Racing movies often include dramatic scenes where racers rev their engine several times and then slowly roll back to their original position. In the same spirit, we implement an intimidation mode.

B. Obstacle Avoidance

During a lap on the grand challenge course, the obstacle is placed on the path. We handle obstacle avoidance within our planner.

C. EKF Lidar

The Extended Kalman Filter (EKF) [1] is a Bayesian localization algorithm used for any system that changes and can be measured. Estimating a system's state with an EKF has been done for a wide array of systems, such as ECG signals [2], satellite pose estimation [3], and robotics [4]. We use odometry and LiDAR scans from the Deepracer to localize.

As the name suggests, the EKF extends the accuracy of the Kalman Filter [5] by applying a linearization of the dynamics and observation model through the use of a Taylor Series approximation to both models. This changes the Kalman filter by swapping the dynamics and observation models with the Jacobian of those matrices. After linearization, the EKF assumes Gaussian noise is introduced by both the motion and measurement model.

D. Course Completion

Given a predestined route, a motion planning algorithm is implemented to navigate and complete the route successfully.

Fig. 1. C-Space of the final demonstration course. The green line indicates the starting line, and the red line is the finish line.

E. Vision Based Depth Estimation of Obstacles

Using only the onboard RGB cameras (no LiDAR), we leverage state-of-the-art machine learning techniques to detect potential obstacles and estimate their depth from the DeepRacer. We then provide a visualization of the model output, allowing the team to assess the performance of the system. This challenge is mainly a proof-of-concept and is an opportunity to apply cutting-edge machine learning to the domain of robotics. This challenge is not used in the final race solution. The challenge was inspired by Tesla's vision-first approach to autonomous driving.

III. SYSTEM

The DeepRacer boasts modest hardware components for sensing, actuation, and computation. The onboard compute is provide by an Intel Atom Processor with 4GB of RAM on Ubuntu 20.04 LTS. For sensing, there are two front facing HD cameras with RGB, an IMU, and a 2D lidar scan. To drive, the DeepRacer uses a brushless motor for all wheel drive powerd by a 7.4V lithium-polymer (LiPo) battery.

IV. METHOD

A. EKF Lidar

We implement an EKF offline for localization. The EKF predicts the vehicle pose given a vehicle kinematics model, a bicycle model, and a vehicle sensor model for 2D LiDAR scans.

As Described in Fig 2, the EKF is composed of two parts, the prediction step and the update step. In the prediction

step the linearized motion model, $g(u, m_{t-1})$, estimates the pose of the robot and modifies the associated uncertainty through the covariance matrix, Σ. Notice the Gaussian noise introduced the motion model through R_t.

The measurement step updates the robot's pose via a linearized measurement model, H. The measurement step calculates Kalman gain, K, which balances the accuracy of observations compared to the covariance matrix from the prediction step.

$$\begin{aligned}
\bar{\mu}_t &= g(u_t, m_{t-1}) \\
\bar{\Sigma}_t &= G_t \Sigma G_t^T + R_t \\
K_t &= \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1} \\
\mu_t &= \bar{\mu}_t + K_t(z - h(\bar{\mu}_t)) \\
\Sigma_t &= (I - K_t H_t)\bar{\Sigma}_t
\end{aligned} \tag{1}$$

Fig. 2. The Extended Kalman Filter Algorithm

Iterative closest point (ICP) [6] is an algorithm that returns the transformation matrix required to fit a source point cloud to a target point cloud. We use ICP between the scans at time t and $t-1$, which gives an estimation of the difference between a current observed view and a "known" set of landmarks. In the EKF, ICP to produces a measurement residual, $z - h(\bar{\mu})$, updating the robot's pose.

B. Course Completion

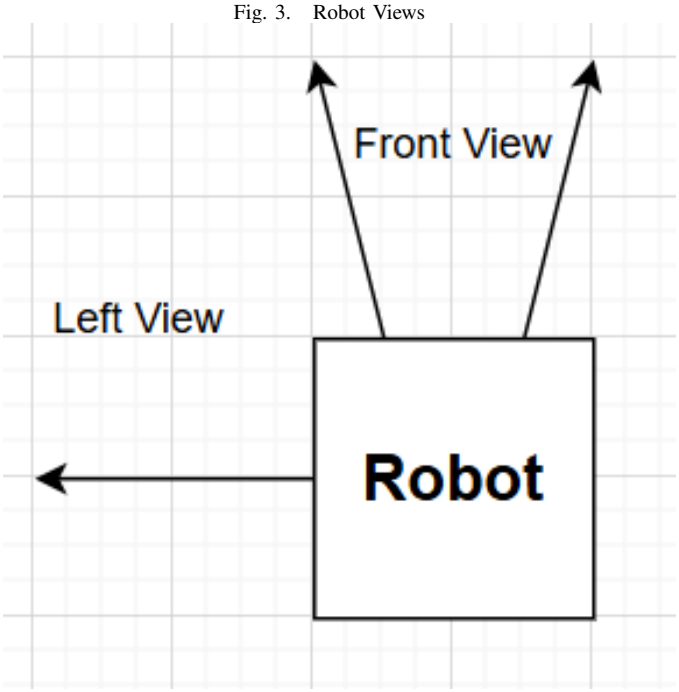
To successfully navigate the course, we implemented a bug algorithm utilizing a PID controller to maintain a set distance from the wall. We deconstructed the Lidar scan message to create a frustum of max 90 degrees to min 60 degrees to the left from the front of the lidar. We then took the median of these values and used that to estimate the distance from the wall. The PID controller stabilized the oscillation of the robot creating a more direct and desired route.

C. Course Completion: Bug Algorithm

The bug algorithm is one of the simplest path-planning algorithms. In this implementation, a lidar collected distance and angle data of the robot's surroundings. This data was then sectioned off into 2 discrete sections, the forward view and the left view. The forward view represents a 30-degree field of view in front of the robot, while the left contained the data between 30 degrees and 90 degrees. The forward view provided data to enable emergency stopping to avoid obstacle collisions, while the path planning relied on the left view.

For each view, only the median value was used for decision-making. The bug algorithm itself turns fully to the left when the median value of the left view is greater than a threshold and fully to the right when it is less than a different threshold. The goal being to keep the robot within a certain distance of the wall. This control has many major drawbacks, chiefly that the velocity vector of the robot is not controlled compared to the wall, only the distance. This allows for the angles to become more and more extreme as algorithm continues to run with the result being the robot running into the wall.

Fig. 3. Robot Views



D. Course Completion: Bug Algorithm Enhanced

The enhanced bug algorithm was implemented to explore the performance of the bug algorithms without implementing higher-level control. This consisted of including more turning angles based on different distances found in the left view. The necessary enhancements to make this approach work would rely on an ever-expanding tree of if statements finally turned for each specific scenario. This quickly became unmanageable ending our pursuit of this control strategy.

E. Course Completion: Range Optimization

The range optimization algorithm was our first attempt at controlling the distance from the left wall as well as the angle. In this algorithm data is collected for both sides of the robot. By adding the median of the left view and the right view we create a range value. When the algorithm starts an initial range value is found. Once the range is found the algorithm aims to keep the robot in at a certain position within that range. This process provides an automatically generated variable turn angle. The turn angle is then compensated by the current range compared to the initial range. This compensation is meant to push the robot to stay parallel to the hallway. While this algorithm did perform better than the previous it was not able to complete the course.

F. Course Completion: Line Best Fit

The line of best fit was the first algorithm we tested that returned promising results. This algorithm took the left view but instead of only using the median created a line of best fit. Then the inverse tangent of the slope was taken; this

$$u(t) = K_p e(t) + K_i \int_0^t e(t)\,dt + K_d \frac{de}{dt} \tag{2}$$

Fig. 4. PID Control Equation

value is the angle that the velocity vector is to the left wall. The angle (in radians) could then be passed directly into the control algorithm. This is due to 1 radian being 57 degrees, which is close to our max steering angle. This control worked extremely well at driving straight down a hallway. Unfortunately, corners were not able to be detected. We still believe that with some more work this approach could return promising results as it was by far the best at driving straight without over correcting.

G. Course Completion: Bug PID

During testing, we observed an unstable behavior turning and path corrections. As a result, a PID controller, Eq 4, was implemented to stabilize the motion of the deepracer [7]. The measurement used was the lidar median value of our left frustum and the control output was the steering angle. Through a trial and error process, we determined the best set-point was 1.5m away from the left wall with gain values set at Kp = 0.8, Ki = 0.3, and Kd = 0.05. Additionally, the PID control program clipped the output values from -0.5 to 0.5 to prevent overturning. In the event of the deepracer turning into a doorway rather than hallway we implemented a stop, reverse, and turn function to correct the deepracer back to the hallway.

H. Course Completion: ROS NAV Stack

The ROS NAV2 stack provides a wide range of capabilities including mapping and localization. Our goal was to smoothly drive the robot around the test track using a tele-op controller to create a map. Then at the demo, the previously generated map would be loaded in, allowing us to run a localization algorithm like a particle filter. With the ROS NAV2 stack and the slam toolbox, this seemed trivial. Unfortunately, due to the nature of open source software and robotics wide range of hardware configurations, the nav stack and slam toolbox were able to be installed were not able to be ran. After hours of debugging without any informative logs there was no clear path forward. This brought our plans of gathering a map and running localization to a close.

I. Vision Based Depth Estimation of Obstacles

Initially, we tested the feasibility of running the machine learning algorithms on the DeepRacer itself. We tried a test run with Midas 2.1 Small [8] (21 million parameters, float 32) using the TensorFlow Lite runtime. Even though the TensorFlow Lite runtime is optimized for resource-constrained devices such as the embedded linux computer on the DeepRacer, the model still took about 6-7 seconds to run inference on a single frame of video. This latency was too slow for even the most trivial of workloads. Another issue we had was the lack of up to date software on the DeepRacer. The DeepRacer uses Ubuntu 20.04 LTS as its operating system, which was released in 2020. This made it hard for us to use state of the art (2024) models and software. So we needed a new plan.

With this in mind, we decided to offload the computational workload to a more capable desktop computer. We chose a Linux desktop in the robotics lab which had access to an NVIDIA RTX 4070 GPU (16GB VRAM). Communication between the DeepRacer and the desktop is facilitated by using the requests python library on the DeepRacer and a machine learning server running on the desktop. The DeepRacer runs a ROS2 subscriber that listens to the camera stream topic and sends API requests to the desktop with the current camera frame in an HTTP form field. The requests go over a Wireguard [9] based VPN managed by Tailscale so that the DeepRacer is able to make requests to the desktop, even in the case when we change WiFi access points when the DeepRacer is moved throughout the Engineering Center. This solved the inference latency and software version issues.

The choice of web framework was mainly determined by which frameworks team members already had experience with. Members of the group already had experience with Python ASGI web frameworks such as FastAPI and Starlette so going in this direction was a natural choice. Starlette was chosen over FastAPI because we wanted to reduce as much overhead as possible. Due to the simplicity of this machine learning service, FastAPI's type checking via Pydantic and automatic OpenAPI documentation would be unnecessary while increasing the latency of the system.

Now that we have a web framework for our machine learning service we need a compatible web server to run it. Since Starlette is implemented with the ASGI (Asynchronous Server Gateway Interface) standard, we needed to choose an ASGI server instead of the typical WSGI (Web Server Gateway Interface) of typical Python web applications. We chose Uvicorn for our web server due to its proven high performance. Uvicorn has been shown to handle higher request throughput with less memory usage compared to alternative ASGI servers such as Daphne and Hypercorn [10].

The model pipeline implemented in the web application is fairly straightforward. The image stream from the DeepRacer is passed to two different models in sequence: an object detector and a depth estimator. The object detection routine forwards the image data though the network, processes the models outputs and returns the image with draw bounding boxes, the labels of detected objects, the indexes of the bounding boxes and the likelihood (score) of the object being detected. The depth estimation routine takes the image data and returns 2 images, one image for visualizing the output, and the other is the predicted depth in meters. We then fuse the data from the two image processing routines by indexing the pixels of the detected object in the depth map and taking the 10%-tile within the bounding box to be the estimated distance of the object. We take this information and produce our visualization with matplotlib which contains the image with drawn bounding boxes, a depth map, and a list of objects

detected and their estimated distance from the camera. This matplotlib figure is then serialized as a jpeg file and returned to the deep racer.

For the object detection model, we used MobileNet v3 (large) [11][12][13] implemented by the torchvision library (3.4 million parameters, float 32). This model was trained on the Microsoft COCO dataset and is very computationally efficient due to its small parameter count. For the depth estimation model we used Depth Anything v2 (small, indoor, metric)[14] implemented by the Hugging Face Transformers library (24.8 million parameters, float 32). This model was trained on the Apple Hypersim (synthetic) dataset. We chose the small version of the model after testing all three versions (small, base, large) and noticing that there was no significant difference in performance between models for our use case (DeepRacer camera).

V. RESULTS

A. Intimidation Mode

The program ran for this challenge consisted of a simple start/stop function. When ran the deepracer jumped forward three times before returning to the start. As expected, the competition was terrified.

B. Obstacle Avoidance

The obstacle avoidance was performed using the same bug algorithm with PID control. Utilizing the stop, reverse, and turn function, it recognized the obstacles but could not navigate back to the track to continue forward. Lowering the speed would yield positive results since it would allow more time for the program to correct itself. Higher speeds led to more problems, as the deep racer would run into the obstacle before stopping and would have less time to react after navigating past it.

C. Course Completion

For the final course competition, the objective was to complete the preset route faster than the other teams. Each team was given three time trial opportunities and then the fastest was used to compare with the opponents. We ran the bug algorithm with PID control as it yielded the most consistent results. The results of the PID controller during the trial are presented in Fig 5. The larger spikes in the Process Variable (PV) vs Setpoint represent the corners of the course and the smaller changes in PV represent the doorways we navigated around.

If we had more time, further testing could have been performed to fine-tune the PID gain values. As observed in the control output, the system could have been more stable and we could have reduced some of the instability in the turns. Additionally, battery fluctuation led to varying results in turning angle and speed values, further complicating troubleshooting and testing of the deepracer's bug algorithms.

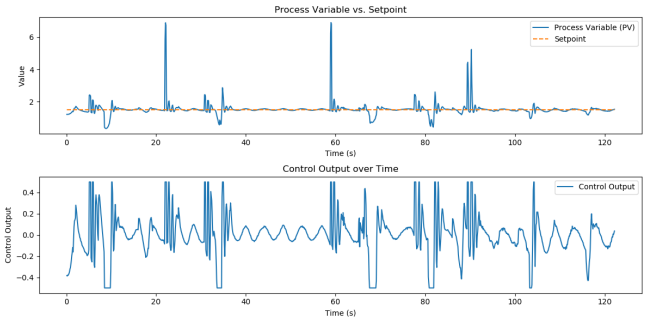


Fig. 5. PID Control through course

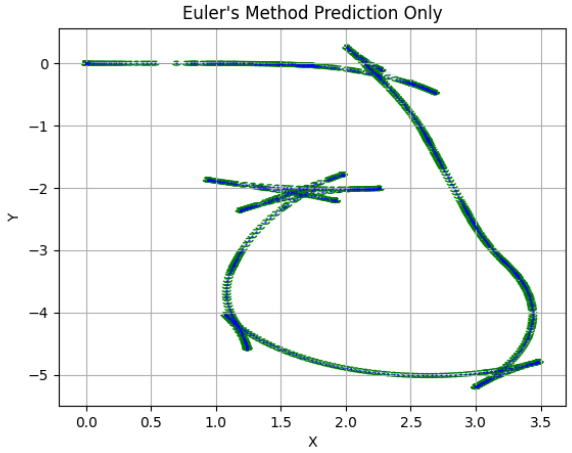


Fig. 6. Applying only the prediction step with the Non-linear Bicycle Dynamics model produces an estimation of the ground truth path.

D. EKF Lidar

To demonstrate the value of each of the terms in EKF, we perform a small ablation over the algorithm's components. We collect a small bag where we teleoperate the DeepRacer around an obstacle, then visualize the localization with non-linear prediction step only, and non-linear prediction with linearized update.

Figure 6, offers the best visual representation of the ground truth path. We introduce the update step by incorporating the linearized measurement model in figure 7

E. Vision Based Depth Estimation of Obstacles

In terms of achieving the objectives as stated in the challenge statement, this implementation of vision based depth estimation of obstacles was a success. We were able to provide a fully functioning demo which made the obstacle and depth estimation information available to the DeepRacer. This system is capable of running online (from the real time camera stream) despite the resource constraints of the DeepRacer itself.

Notable areas of improvement of the system include:

- The depth estimation is not accurate. We did not empirically test this but we "eyeballed" that Depth Anything would consistently overestimate the true distance by

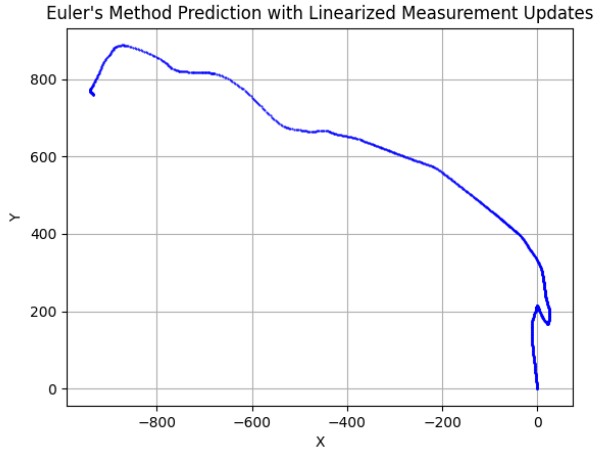


Fig. 7. With a linearized measurement update, the robots path is affected by the residuals between the previous scan and the current scan.

about +1 meter. This can be mitigated by retraining the model on better training data.

- The depth estimation logic struggled when the environment was dense, or had overlapping bounding boxes. We tried to mitigate this issue by taking a quantile of the depth map within the bounding box, but it is not guaranteed to fix this issue for any case. In the future, we could try using semantic segmentation so we never index any pixels that are not the detected entity.
- Running even these "small" machine learning models can be computationally infeasible for many robotic systems, of which the DeepRacer is definitely one. This may cause a barrier to adoption for industry use cases. In cases where proper hardware such as a GPU can be provided, the added hardware would certainly drive up the cost of purchasing and maintaining the product, which could also prevent adoption by some robotic companies. We can attempt to mitigate this by using performance optimization techniques on our models.

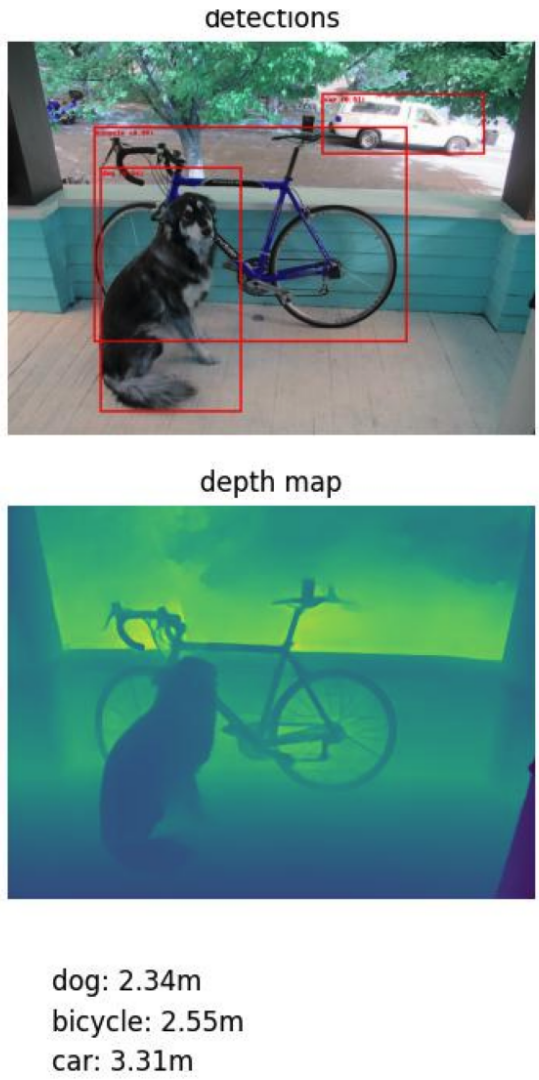
VI. CONCLUSIONS

In this report, we demonstrated the capabilities of an AWS deepracer to navigate a course by implementing a bug algorithm augmented with a PID controller for precise wall-following. This approach enabled the vehicle to successfully traverse the environment while keeping an optimal distance away from the wall. Additionally, we explored advanced sensing methodologies including EKF lidar for state estimation and vision-based depth estimation for object detection. These techniques provided insights into the robot's surroundings, enhancing its navigational accuracy.

ACKNOWLEDGMENT

Jack Center, Doncey Albin, and Destin Woods

Fig. 8. Visualization acquired from a test image. The depth model seems to struggle with the detected car.



REFERENCES

[1] M. I. Ribeiro, "Kalman and extended kalman filters: Concept, derivation and properties," *Institute for Systems and Robotics*, vol. 43, no. 46, pp. 3736–3741, 2004.

[2] O. Sayadi and M. B. Shamsollahi, "Ecg denoising and compression using a modified extended kalman filter structure," *IEEE transactions on biomedical engineering*, vol. 55, no. 9, pp. 2240–2248, 2008.

[3] N. Filipe, M. Kontitsis, and P. Tsiotras, "Extended kalman filter for spacecraft pose estimation using dual quaternions," *Journal of Guidance, Control, and Dynamics*, vol. 38, no. 9, pp. 1625–1641, 2015.

[4] S. Thrun, "Probabilistic robotics," *Communications of the ACM*, vol. 45, no. 3, pp. 52–57, 2002.

[5] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.

[6] K. S. Arun, T. S. Huang, and S. D. Blostein, "Least-squares fitting of two 3-d point sets," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-9, no. 5, pp. 698–700, 1987.

[7] S. Bennett, "Development of the pid controller," *IEEE Control Systems Magazine*, vol. 13, no. 6, pp. 58–62, 1993.

[8] K. Lasinger, R. Ranftl, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *CoRR*, vol. abs/1907.01341, 2019. [Online]. Available: http://arxiv.org/abs/1907.01341

[9] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security*

Symposium (NDSS). San Diego, CA, USA: The Internet Society, February 2017, draft Revision as of June 1, 2020. Permanent ID: 4846ada1492f5d92198df154f48c3d54205657bc. [Online]. Available: https://www.wireguard.com/papers/wireguard.pdf

[10] A. Novikau, "Evaluative comparison of asgi web servers: A systematic review," *International Journal of Science and Engineering Applications*, vol. 13, no. 03, pp. 30–35, 2024. [Online]. Available: https://www.ijsea.com

[11] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. E. Reed, C. Fu, and A. C. Berg, "SSD: single shot multibox detector," *CoRR*, vol. abs/1512.02325, 2015. [Online]. Available: http://arxiv.org/abs/1512.02325

[12] M. Sandler, A. G. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation," *CoRR*, vol. abs/1801.04381, 2018. [Online]. Available: http://arxiv.org/abs/1801.04381

[13] A. Howard, M. Sandler, G. Chu, L. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for mobilenetv3," *CoRR*, vol. abs/1905.02244, 2019. [Online]. Available: http://arxiv.org/abs/1905.02244

[14] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," 2024. [Online]. Available: https://arxiv.org/abs/2406.09414

APPENDIX

Here we detail the contribution of each of the authors.

A. Miles Mena

- Implemented intimidation mode
- Collected robot paths
- Read robot data offline
- Implemented prediction step
- Linearized observation model
- Iterative closest point
- Debugging planning algorithm

B. Nick McConnell

- Implemented Bug Algorithm
- Lidar frustum testing
- Algorithm Prototyping
- PID implementation and tuning
- Debugging planning algorithm

C. Spencer Mullinix

- Implemented Linearized EKF prediction
- Visualized Iterative Closest Point
- Explored ROS2 NAV Stack
- Algorithm Prototyping
- PID Tuning

D. Jonathan Wu

- Implemented the challenge "Vision Based Depth Estimation of Obstacles" end to end.